



Compilation On The GPU? A Feasibility Study

Robin F. Voetter
LIACS, Leiden University
Leiden, The Netherlands
robin@voetter.nl

Marcel Huijben
LIACS, Leiden University
Leiden, The Netherlands
marcelhuij@live.nl

Kristian F. D. Rietveld
LIACS, Leiden University
Leiden, The Netherlands
k.f.d.rietveld@liacs.leidenuniv.nl

ABSTRACT

The emergence of highly parallel architectures has led to a renewed interest in parallel compilation. In particular, the widespread availability of GPU architectures raises the question whether compilation on the GPU is feasible. In this paper, we describe the first design and implementation of a parallel compiler from a simple imperative programming language to RISC-V machine code, that is fully executed on a GPU. To accomplish this, all stages from parsing to machine code generation were redesigned to exploit fine-grained parallelism. Experimental evaluation of the implemented prototype demonstrates our proposed parallel techniques to be effective and implementation of compilation on the GPU to be feasible. Finally, we propose a number of avenues for future work and hope to revitalize research into parallel compilation conducted in the 1980s.

CCS CONCEPTS

• **Software and its engineering** → **Compilers**; • **Theory of computation** → *Parallel algorithms*.

KEYWORDS

Parallel Compilation, GPUs, Compiler Construction, Parsing, Code Generation

ACM Reference Format:

Robin F. Voetter, Marcel Huijben, and Kristian F. D. Rietveld. 2022. Compilation On The GPU? A Feasibility Study. In *19th ACM International Conference on Computing Frontiers (CF'22)*, May 17–19, 2022, Torino, Italy. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/3528416.3530249>

1 INTRODUCTION

In the 1980's and 1990's parallel compilation was an active area of research [7, 15, 16]. Despite these efforts, a fully parallel compiler has never been realized. Currently, parallelization of the compilation process is typically achieved at a coarse-grained level by processing different source files (translation units) at the same time, or by performing analysis on compilation units in parallel. With the stagnation of the increase in single-core performance [18] and the emergence of highly parallel architectures, compiler design warrants to be reconsidered. In fact, there is a renewed interest in parallel compilation by taking the parallelization to an even finer-grained level [3, 10, 19].

We are in particular interested in the feasibility of performing the entire compilation process on Graphics Processing Units (GPUs), which are now widely available. This might pave the way to accelerate the compilation process by taking advantage of features introduced in hardware in the last years, such as SIMD units and GPUs, or to offload compilation to GPUs contrary to preparing executable GPU code on the CPU beforehand.

In this paper, we present a feasibility study on implementing and executing the full compilation chain on the GPU. More specifically, we describe the first design and implementation of a parallel compiler from a simple imperative programming language to RISC-V machine code. We have devised our own simple programming language due to the limitations of current parallel parsers and the scope of this study. This language serves as a demonstrator that implementation of a fully parallel compiler, by redesigning all compilation stages to exploit fine-grained parallelization, is feasible. To the best of our knowledge, our implementation is the first to perform all of these steps in parallel on single translation units. We extend and adapt prior work on parallel lexing and parsing [7, 15, 20] and describe our implementation of parallel semantic analysis and code generation, including register allocation.

Subsequently, we perform an experimental evaluation to see whether compilation on the GPU is advantageous from a performance perspective. Experiments conducted on a modern GPU architecture show a sub-linear scaling of the execution time against a linear increase in the size of the source file for the majority of the compiler stages. This shows our proposed parallel techniques to be effective. In fact, for large input files better scaling compared to traditional CPU-based compilers is already seen, showing the potential of GPU-based compilation. Based on these initial results, we identify a number of avenues for future work, in particular to improve the register allocation stage which is shown to be a bottleneck. By doing so, there is the potential to lower the bound on the input file size for GPU compilation to be profitable. Furthermore, due to the portable nature of our implementation, the study can be extended to many-core SIMD CPU architectures in the future.

Contributions. Our work makes the following contributions:

- We describe an implementation of a compiler running completely on the GPU, which is capable of taking a program written in a custom designed imperative programming language and compiling it into RISC-V instructions.
- We evaluate the effectiveness of the design and give an indication of the performance compared to traditional compilation with a number of experiments.
- Based on these results, we identify avenues for future work to revitalize research into parallel compilation techniques.
- We have released the compiler as open source, available at <https://github.com/Snektron/pareas>.



This work is licensed under a Creative Commons Attribution International 4.0 License.
CF'22, May 17–19, 2022, Torino, Italy
© 2022 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-9338-6/22/05.
<https://doi.org/10.1145/3528416.3530249>